Types de données en Java

Dany Siriphol – CODA 1 ère année - 2025

Types primitifs

Data Type	Description
`byte`	Stores whole numbers from -128 to 127
`short`	Stores whole numbers from -32,768 to 32,767
`int`	Stores whole numbers from -2,147,483,648 to 2,147,483,647
`long`	Stores whole numbers from -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
`float`	Stores fractional numbers. Sufficient for storing 6 to 7 decimal digits
`double`	Stores fractional numbers. Sufficient for storing 15 to 16 decimal digits
`boolean`	Stores true or false values
`char`	Stores a single character/letter or ASCII values

Types non primitifs

Les types **non primitifs** sont des types de données qui sont inclus dans le langages mais qui sont représentées par des classes. Chaque variable est une **référence** vers un objet

- String
- Array
- Collections
- Class
- Object

Strings

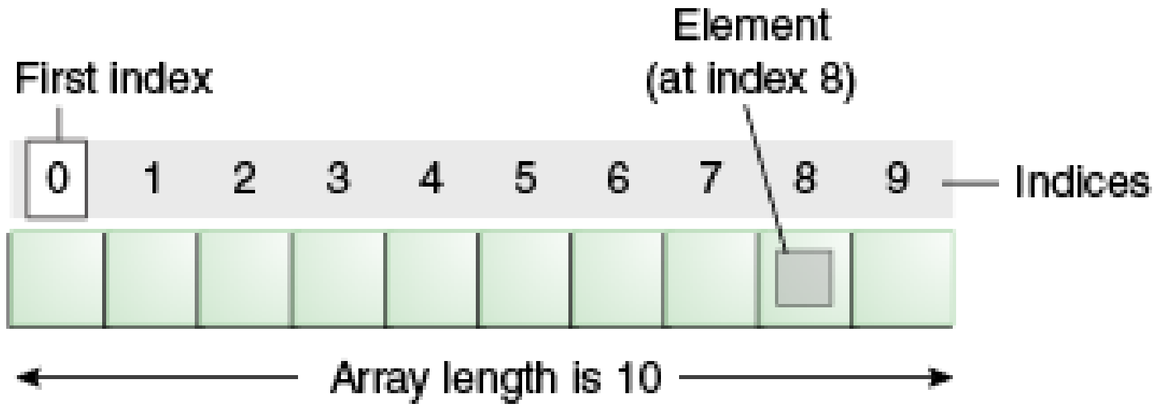
En Java les strings sont des instances de la classe String

```
String str = "java";

String str = new String("Java");
```

Tableaux

Tableaux

Les tableaux permettent de stocker un nombre fixe de données



Déclaration d’un tableau

```
int[] array = new int[10];

int[] numbers = {
    100, 200, 300,
    400, 500, 600,
    700, 800, 900, 1000
};

String[] texts;
```

Utilisation d’un tableau

```
numbers[0] = 100;
numbers[1] = 200;
numbers[2] = 300;

System.out.println(numbers[0]);
```

Parcourir un tableau

```java
int[] numbers = {10, 20, 30, 40, 50};

for (int i = 0; i < numbers.length; i++) {
    System.out.println("Indice " + i + " : " + numbers[i]);
}
```

Arrays : Methodes utilitaires sur les tableaux

```
import java.util.Arrays;
```

```
int[] numbers = new int[5];
Arrays.fill(numbers, 42);
```

Arrays : Méthodes utilitaires sur les tableaux

- Arrays.toString
- Arrays.sort
- Arrays.equals
- Arrays.compareTo
- ...

https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/Arrays.html

Collections

Collections

Java possède une famille de classe que l’on appelle les collections. Ce sont des structures de données qui permet de stocker, manipuler et parcourir un groupe d'objets

Les collections contrairement aux tableaux ont des tailles dynamiques

Elles font partie du package Java.util et il faut les inclure au début du programme

ArrayList

Une ArrayList permet de stocker une liste d'objets. Elle possède de nombreuses méthodes permettant de modifier le contenu de la liste et d'autres fonctions utilitaire. Contrairement aux tableaux, sa taille est variable.

```java
import java.util.ArrayList;

ArrayList<String> names = new ArrayList<String>();
names.add("Alice");
names.add("Bob");
names.add("Alice");
System.out.println(names);  // [Alice, Bob, Alice]
```

ArrayList : méthodes

```
names.get(0);

names.set(0, "John");

names.size();

names.remove("Alice");
names.remove(0);
```

https://docs.oracle.com/en/java/javase//21/docs/api/java.base/java/util/ArrayList.html

HashSet

Un HashSet est une collection dans laquelle chaque élément est unique. Elle possèdes beaucoup de méthodes communes avec l'ArrayList

```
import java.util.HashSet;

HashSet<String> cars = new HashSet<String>();
cars.add("Volvo");
cars.add("BMW");
cars.add("Ford");
cars.add("BMW"); // ce doublon ne sera pas ajouté
cars.add("Mazda");
System.out.println(cars);
```

HashMap

Une HashMap est une collection dans laquelle les éléments sont stockés, non pas par index, mais par des pairs clé/valeurs. On appelle ce genre de collections des **dictionnaires**

```java
import java.util.HashMap;


HashMap<String, String> capitalCities =
        new HashMap<String, String>();

// Add keys and values (Country, City)
capitalCities.put("England", "London");
capitalCities.put("Germany", "Berlin");
capitalCities.put("Norway", "Oslo");
capitalCities.put("USA", "Washington DC");
System.out.println(capitalCities);
```

Dictionnaire : HashMap

Méthodes

```
capitalCities.get("England");

capitalCities.remove("England");

capitalCities.size();
```

Parcourir une collection : boucle foreach

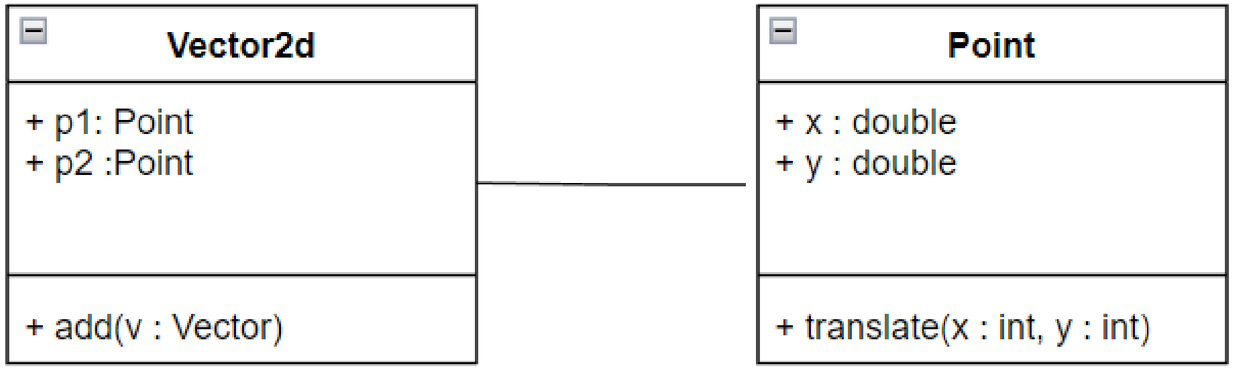
```
for (String s : names) {
    System.out.println(s);
}

for (String car : cars) {
    System.out.println(car);
}

for (String i : capitalCities.keySet()) {
  System.out.println("key: " + i + " value: " + capitalCities.get(i));
}
```

POO : Composition

Composition

Définition

On appelle la composition le fait qu'une classe utilise d'autres classes. Pour cela, une classe doit avoir des **instances** d'une autre classe en **propriété**



Composition

```java
public class Vector2D {
    private Point p1;
    private Point p2;

    public Vector2D(Point p1, Point p2) {
        this.p1 = p1;
        this.p2 = p2;
    }

    public double length() {
        double dx = p2.x - p1.x;
        double dy = p2.y - p1.y;
        return Math.sqrt(dx * dx + dy * dy);
    }
}
```

```java
public class Point {
    public double x;
    public double y;

    public Point(double x, double y) {
        this.x = x;
        this.y = y;
    }
}
```

POO : Instance et fonctions

Dany Siriphol – CODA 1 ère année - 2025

Instance et paramètre de fonction

Syntaxe

Une instance d’une classe peut être passé en paramètre d’une fonction

```
public class Point {
    public double x;
    public double y;

    public Point(double x, double y) {
        this.x = x;
        this.y = y;
    }

    public void add(Point p) {
        this.x += p.x;
        this.y += p.y;
    }
}
```

Instance et retour de fonction

Syntaxe

Une méthode peut retourner une instance d’une classe

```
public Point clone() {
    return new Point(this.x, this.y);
}
```

Passage de paramètres : types primitifs vs non primitifs

En Java :

- les types primitifs sont passés par valeur

```
public void doubleInt(int a) {
    a *= 2; // le paramètre sera modifié dans la fonction mais pas a l'exterieur
}

// ...

int a = 5;
doubleInt(a);
```

Passage de paramètres : types primitifs vs non primitifs

En Java :

- les types non primitifs sont passés par référence

```
// l'instance p va être modifiée
public void movePoint(Point p) {
    p.x += 5;
    p.y += 10;
}
```

POO : propriétés et méthodes statiques

Dany Siriphol – CODA 1 ère année - 2025

Méthode statiques

Définition

Une méthode statique est une méthode qui peut être appelée sans instance

```java
public class Vector2D {
    private Point p1;
    private Point p2;

    public Vector2D(Point p1, Point p2) {
        this.p1 = p1;
        this.p2 = p2;
    }

    public static Vector2D createEmptyVector() {
        return new Vector2D(new Point(0, 0), new Point(0, 0));
    }
}
```

```java
public static void main(String[] args) {

    Vector2D v = Vector2D.createEmptyVector();

}
```

La classe java.Util.Arrays est composé de nombreuses méthodes statiques

Exercices